Studio Nx

Backend(FastAPI/Python) 09 — Inquiry 모델 컬럼

2026-10-12

Backend(FastAPI/Python) 09 — `Inquiry` 모델 컬럼

항목	내용
파일	`studionx/backend/app/models/inquiry.py`
작업	P1-03 백엔드 공통: ORM 모델 컬럼 (`docs/DB.md` §3.1, `studionx/db/01_phase1_account.sql`)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	08 `User` 모델 · 10 `Project` 모델

1. 하는 일

MySQL의 `inquiries` 테이블(문의)을 파이썬 클래스 `Inquiry`로 옮겨 적습니다. 08번 `User`와 같은 방식이며, 이번에 새로 나온 것은 **다른 테이블을 가리키는 칸(FK)**과 **긴 글을 담는 칸(`TEXT`)**입니다.

새로 나온 것	칸	뜻
FK (외래 키)	`user_id` → `users.user_id`	이 문의를 보낸 회원입니다. 손님이면 비어 있습니다
`TEXT`	`content`, `answer`	길이 제한이 거의 없는 글입니다 (`VARCHAR`는 길이를 정해야 합니다)

2. 언제 쓰나

쓰는 곳	하는 일	칸
P1-12 문의 등록 (home.html의 Contact 폼)	문의를 저장합니다	`user_id`(07번의 결과), `email`, `title`, `content`, `privacy_agreed_at`
P1-21 관리자 화면 (SQLAdmin)	목록을 보고 답변합니다	`status`, `answer`, `answered_at`

문의 상태 (`status`)	뜻
`RECEIVED` (기본)	접수되었고, 아직 답변이 없습니다
`ANSWERED`	관리자가 답변했습니다
`CLOSED`	처리가 끝났습니다

3. 코드

import (파일 맨 위)

```
from datetime import datetime
from enum import StrEnum

from sqlalchemy import BigInteger, Enum, FetchedValue, ForeignKey, String, Text, text
from sqlalchemy.dialects.mysql import DATETIME
from sqlalchemy.orm import Mapped, mapped_column

from app.db.session import Base
```

`BigInteger`, `Mapped`, `mapped_column`, `Base`는 원래 있었고, 나머지를 이번에 추가했습니다. 08번과 같으며, **`ForeignKey`와 `Text`** 두 개가 새로 들어갔습니다.

import	가져오는 것	쓰는 곳
`ForeignKey`	다른 테이블의 칸을 가리키는 표시	`user_id`
`Text`	SQL의 `TEXT`	`content`, `answer`
나머지	08번의 표와 같습니다	

함수 (클래스와 칸 선언)

```
class InquiryStatus(StrEnum):
    RECEIVED = "RECEIVED"
    ANSWERED = "ANSWERED"
    CLOSED = "CLOSED"


class Inquiry(Base):
    __tablename__ = "inquiries"

    inquiry_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=True)

    user_id: Mapped[int | None] = mapped_column(BigInteger, ForeignKey("users.user_id"))
    name: Mapped[str | None] = mapped_column(String(50))
    email: Mapped[str] = mapped_column(String(255))
    title: Mapped[str] = mapped_column(String(200))
    content: Mapped[str] = mapped_column(Text)
    privacy_agreed_at: Mapped[datetime] = mapped_column(DATETIME(fsp=3))
    status: Mapped[InquiryStatus] = mapped_column(
        Enum(InquiryStatus), server_default=InquiryStatus.RECEIVED
    )
    answer: Mapped[str | None] = mapped_column(Text)
    answered_at: Mapped[datetime | None] = mapped_column(DATETIME(fsp=3))
    created_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
    )
    updated_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)"), server_onupdate=FetchedValue()
    )
```

SQL과 비교 (08번에 없던 것만)

SQL (`01_phase1_account.sql`)	파이썬
`user_id BIGINT UNSIGNED NULL` + `FOREIGN KEY (user_id) REFERENCES users (user_id)`	`Mapped[int | None]`, `ForeignKey("users.user_id")`
`content TEXT NOT NULL`	`Mapped[str]`, `Text`
`KEY idx_inquiries_status (status)` (검색용 색인)	적지 않았습니다 (아래 §5 참고)

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름`만 가져와서 그대로 씁니다

식	뜻
`ForeignKey("users.user_id")`	"이 칸의 값은 `users` 테이블의 `user_id` 중 하나"라는 표시입니다. `"테이블.칸"` 형식의 문자열로 적습니다
`Mapped[int | None]` + FK	비어 있어도 됩니다(손님 문의). 값이 있으면 실제 회원 번호여야 합니다
`Text`	길이를 적지 않습니다. MySQL의 `TEXT`는 약 64KB(한글 약 2만 자)입니다

4. 흐름 (손님 문의 하나를 넣을 때)

1. `Inquiry(email=…, title=…, content=…, privacy_agreed_at=…)`처럼 4칸만 넣어서 만듭니다.
2. `db.add` → `db.flush()`로 `INSERT`합니다.
3. DB가 `inquiry_id`, `status=RECEIVED`, `created_at`을 채웁니다. `user_id`는 비어 있습니다.
4. 로그인한 회원이라면 1단계에서 `user_id=7`을 함께 넣습니다. 없는 번호라면 DB가 거절합니다.

5. 왜 이렇게

선택	이유
`user_id`를 비울 수 있게 함	손님도 문의할 수 있기 때문입니다 (FR-112). 07번이 `None`을 넘기면 그대로 비워 둡니다
FK를 파이썬에도 적음	테이블은 SQL이 만들지만, 적어 두면 나중에 `relationship`(문의 → 회원)을 붙일 수 있고, 코드만 봐도 연결을 알 수 있습니다
색인(`idx_inquiries_status`)은 적지 않음	색인은 DB가 검색을 빠르게 하려고 가진 것이라서, 파이썬에서 읽고 쓰는 데는 필요하지 않습니다. 08번도 마찬가지입니다
`content` 최대 2,000자 검사는 여기서 하지 않음	칸(`TEXT`)은 2,000자보다 훨씬 큽니다. 글자 수 검사는 입력 스키마(P1-12, 02번의 422)가 합니다
`name` 칸도 적음	폼에서는 받지 않는 예비 칸이지만, 테이블에 있으므로 모델도 같게 맞춥니다
회원이 탈퇴해도 문의는 남김	FK에 `ON DELETE`가 없습니다. 탈퇴는 행을 지우지 않고 `status=DELETED`로 바꾸는 방식이므로(08), 문의의 `user_id`도 그대로 남습니다

6. 시험

`tests/test_inquiry_model.py` **(새 파일, 5개)**: 실제 MySQL과 비교했습니다. 넣은 행은 시험이 끝나면 되돌립니다 (시험 뒤 확인: 0행).

시험	확인한 것
칸이 DB와 같은지	칸 12개의 이름, 순서, `NULL` 허용 여부, FK(`user_id` → `users.user_id`)가 실제 테이블과 같습니다
손님 문의	4칸만 넣으면 `user_id`는 비어 있고, `status`는 `RECEIVED`이며, `created_at`은 채워지고, 답변 칸은 비어 있습니다
회원 문의	회원을 만들고 그 번호를 넣으면 저장됩니다
없는 회원 번호 (999999999)	DB가 거절합니다 (`IntegrityError`)
2,000자 글	그대로 저장되고 다시 읽힙니다

`tests/conftest.py` **정리 (모든 모델 시험이 함께 씁니다)**

추가한 것	하는 일
`db`	실제 MySQL 세션입니다. 연결할 수 없으면 시험을 건너뛰고, 끝나면 되돌립니다(rollback)
`assert_matches_db(모델)`	모델의 칸 이름, 순서, `NULL` 허용 여부, FK를 실제 테이블과 비교합니다

08번의 `tests/test_user_model.py`에 있던 같은 코드를 여기로 옮겼고, 08번 시험도 이것을 쓰도록 바꿨습니다. 남은 모델(10번부터)도 이것을 씁니다.

전체 시험 57개가 통과했고, `ruff` 검사도 통과했습니다.